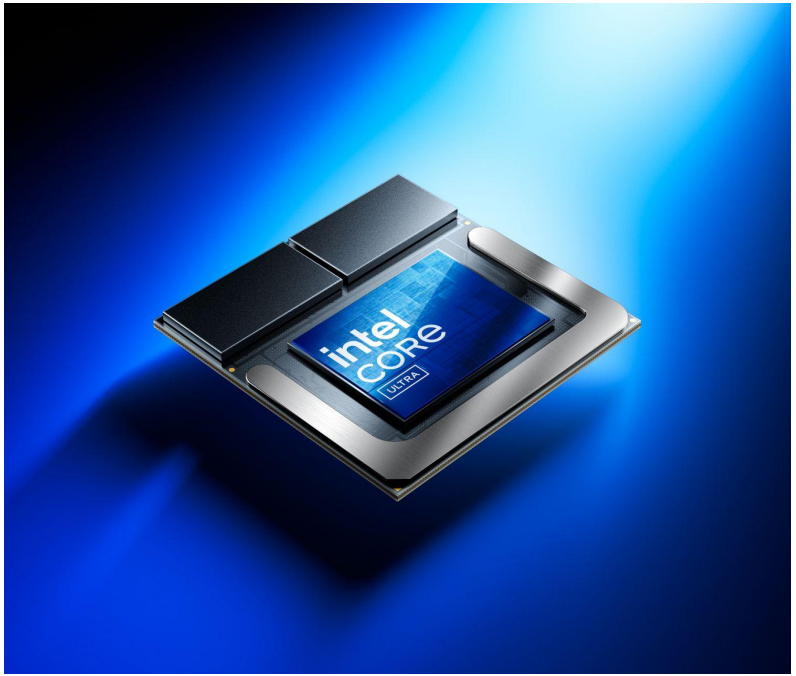
Intel Lunar Lake

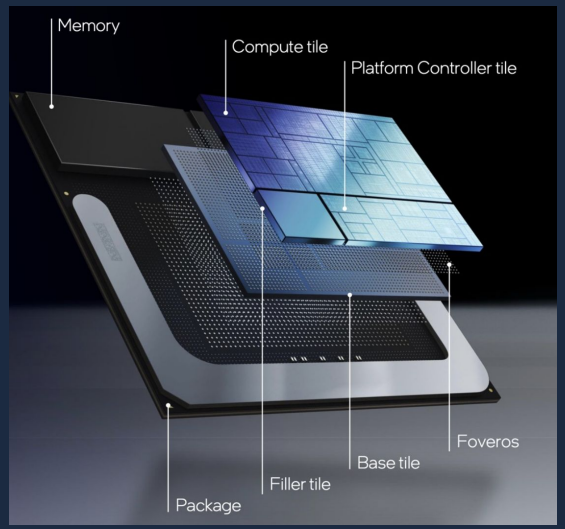
Team: Sai, Valli, Herbie

Lunar Lake's Overview and ISA

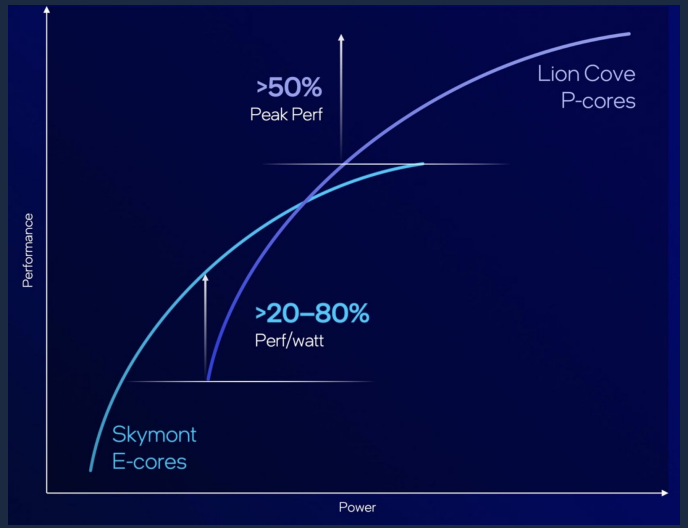
- Low power processor based on intel's X86 design.
- 8 cores - 4 performant cores (Lion Cove P Cores) and 4 low power efficient cores (Skymont E cores).
- On package LPDDR5 DRAM
- P core max turbo frequency of 5.1 GHz and E core max turbo frequency of 3.7 GHz.
- CMPccXADD instructions and SW Resource prioritization.
- No hyperthreading
- Total Store Order (TSO)



Intel Lunar Lake



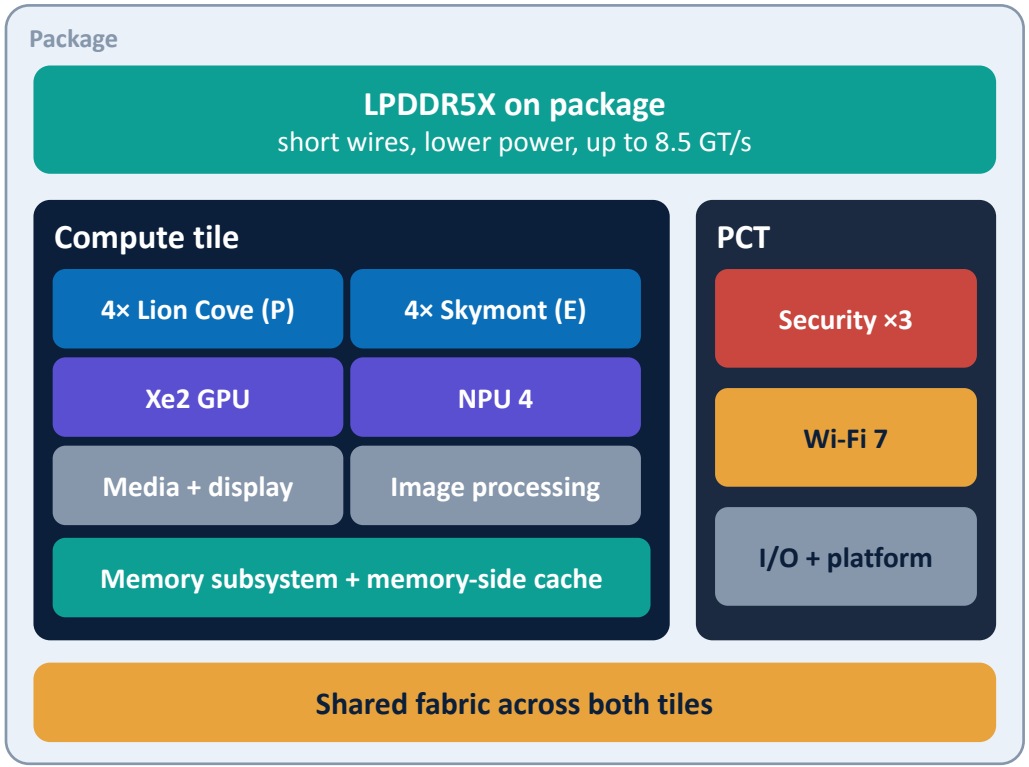
Compute tile and platform controller tile sit on a Foveros base tile, with memory on the package.



Intel's illustration: E-cores are 20–80% more efficient at low power, and P-cores reach over 50% higher peak performance.

Images: Intel Tech Tour

Lunar Lake package: two tiles on one fabric



Why split into tiles

Leading-edge process only where it pays off, an older process for I/O, and smaller dies yield better.

Why DRAM on package

Short wires give lower latency and much lower interface power.

Extended scalability

Both tiles share one fabric, so designers can move IPs between dies.

Specialized units outside the cores

Xe2 GPU, media, display

- XMX engines: matrix math for AI
- Media engine decodes H.266 (VVC)
- 3 display pipelines: HDMI 2.1, DP 2.1, low-power eDP 1.5

NPU 4

- 6 NPUs + 12 SHAVE DSPs
- 9 MB cache
- On the compute tile, next to the GPU

Platform controller tile

- Security: 3 separate engines, each with its own tasks
- Connectivity: integrated Wi-Fi 7
- Same fabric as the compute tile

PMIC and power

- Fixed and variable rails (DRAM is variable)
- E-cores: own rail, gated off with their cache
- Per-rail power telemetry reported back
- AI self tuning controller

ISA extensions

Both the E and P cores support the following common extensions:

- SSE, SSE2, SSE3, SSSE3, SSE4.x
- AVX2
- Many more

This allows the cores to process SIMD (Single instruction multiple data) instructions.

E cores support 128 bit SIMD instructions.
P cores support 256 bit SIMD instructions.

On a P core, we could do 8, 32 bit integer operations at once such as multiplying 8 integers by a constant.

Lunar lake also added the relatively new CMPCCXADD which allows the CPU to evaluate a condition and perform math in a single, atomic step instead of bouncing between threads in a while loop for multithreaded applications.

Front end - P core (Lion cove)



FETCH and DECODE Stages

- BPU decides where to start fetching next through the I-TLB (256 entry, 8 way) and I-Cache (64KB, 16 way) into an instruction queue. Then:

If its a simple x86 instruction:
- Goes through normal decode
- Feeds into uOP queue 8 wide

If its a very complex x86 instruction:
- Goes through Microcode Sequencing ROM
- Feeds into uOP queue 4 wide

If it's already in uOP Cache
- Feeds into uOP queue 12 wide

uOP Cache:
- Holds recently translated x86 instructions in uOP form.
- Approximately 5,250 uOPs.

uOP queue:
- Holds instructions ready for the reorder stage to process
- 192 Entries

Source: Chips and Cheese's independent analysis

Front end - P core (Lion cove)

Branch Prediction Unit (BPU)



Branch target buffers:
- Predict where to start fetching next
- 3 levels with different delay
- Each entry = one branch

Fastest: 256 Entries - 0 cycle
Medium: 6K Entries - 2 cycle
Slowest: 12K Entries - 3/4 cycle

Branch prediction:
- Predict taken vs not taken
- Intel doesn't disclose the algorithm

A repeating pattern of 12,000 outcomes shows very little evidence of a prediction penalty in independent testing.

Impressive but nearly on par with other modern CPUs.

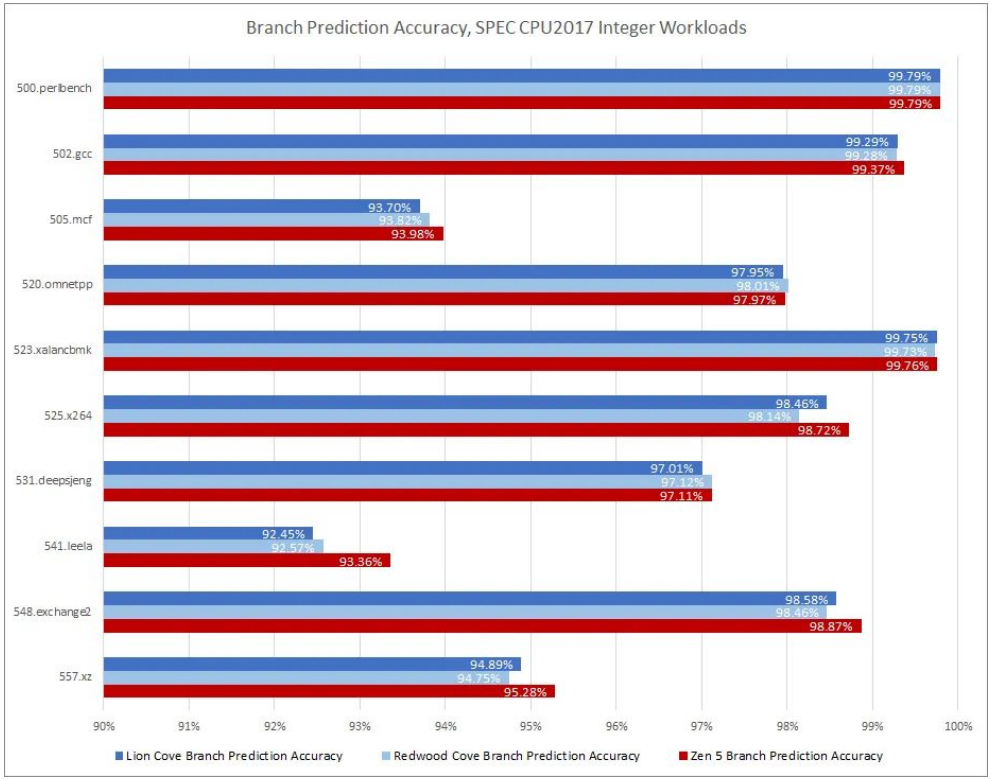
Return stack:
- Used to predict where to start fetching next after a function returns
- 2 levels

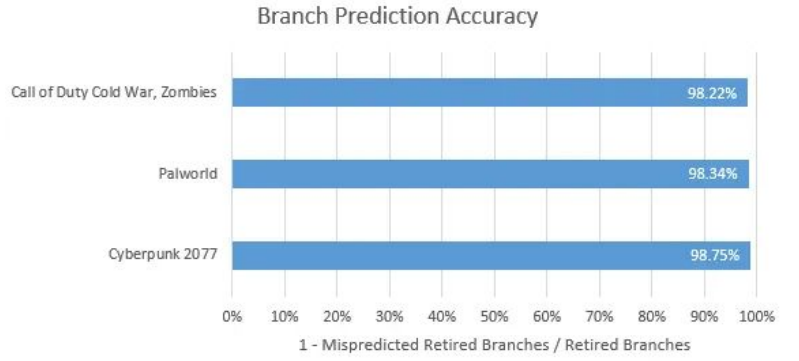
Fastest: 12 Entries
Slowest: 24 Entries

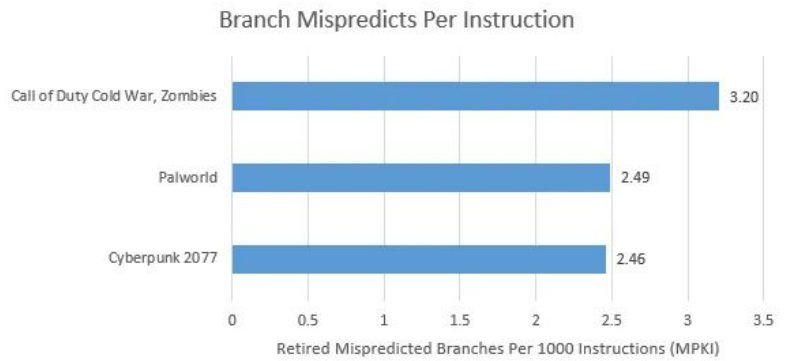
Source: Chips and Cheese's independent analysis

Front end - P core (Lion cove)

Branch Prediction Stats







Source: Chips and Cheese's independent analysis

Front end - P core (Lion cove)



RENAME Stage + Register file

Register Alias Tables:

- Maps x86 architectural registers to physical registers
- 8 Wide
- Instructions sent to Reorder buffer and Branch order buffer (speculated to be used for branch recovery)

Integer register set:
274 physical registers

Vector/FP register set:
390 physical registers

MMX/x87 register set:
158 physical registers

Zero Idiom :

- An instruction that sets a register to 0 is optimized here.

Move elimination :

- Register to register moves are removed and just mapped

Source: Chips and Cheese's independent analysis

Back end - P Core (Lion cove)

Reorder buffer and Schedulers

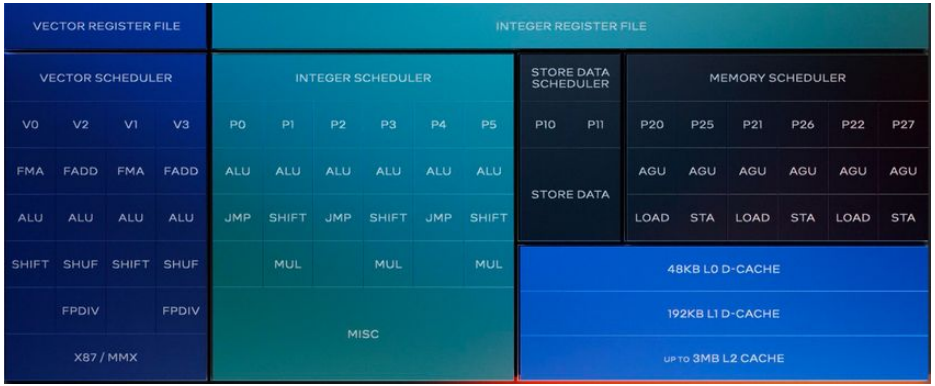
- Reorder buffer holds 576 instructions. It tracks in flight instructions and retires up to 12 instructions per cycle in order.
- The OOO Engine is split into Integer and Vector domains (Main change from last generation)
- Connected to 4 different schedulers which feed into functional units:

Integer scheduler:
- Integer arithmetic
- Branches

Feeds into
6 total integer units

Vector scheduler:
- SIMD/Vector
- Floating Point
- FMA

Feeds into
4 total vector/FP units



Source: Chips and Cheese's independent analysis, and Intel

Back end - P Core (Lion cove)

More schedulers, Load/Store

Memory scheduler:
- Memory operations
- Loads
- Calculate store addresses

Feeds into
3 Load, 3 Store AGUs

Store data scheduler:
- Store operations
- Calculate store data

Feeds into
2 Store data Units

load queue (189 entry)
store queue (120 entry)

These two connect to the AGUs and store data units.

Together they ensure:

1. Prevention of Load Store hazards
2. Buffer memory operations while they wait to complete or become eligible to commit.
3. Supports the Total Store Order that x86 requires (loads can be reordered ahead of old stores if the memory address is different)

When the corresponding instruction retires at the ROB, the queued store could move to the L0 Cache.

Source: Chips and Cheese's independent analysis

Back end - P Core (Lion cove)

Instruction dispatch / issue

1. After register renaming, uOPs are dispatched into the ROB which keeps track of completion

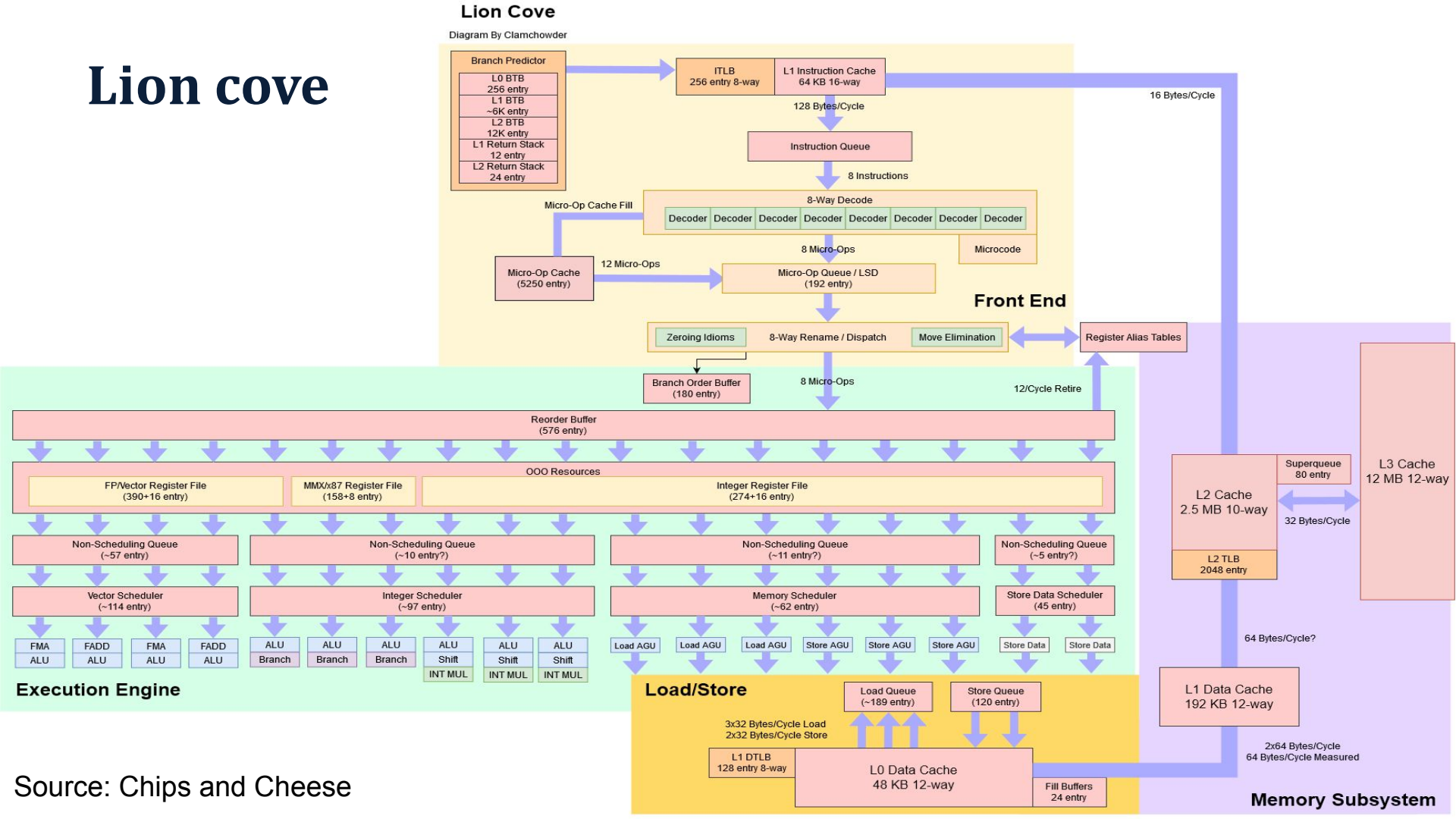
2. uOPs go into the appropriate schedulers and is woken up to execute once:
   - Functional unit is free
   - Dependencies have written back

3. Up to 12 executed instructions can retire at once
   - They are all in program order
   - None of them are on a speculated path

4. Register Alias Table is updated for register renaming.

Exact details are not given by intel such as how and when a value in the store buffer write to the memory subsystem after a retire or how exactly a rollback happens for precise exceptions.

Lion cove



Source: Chips and Cheese

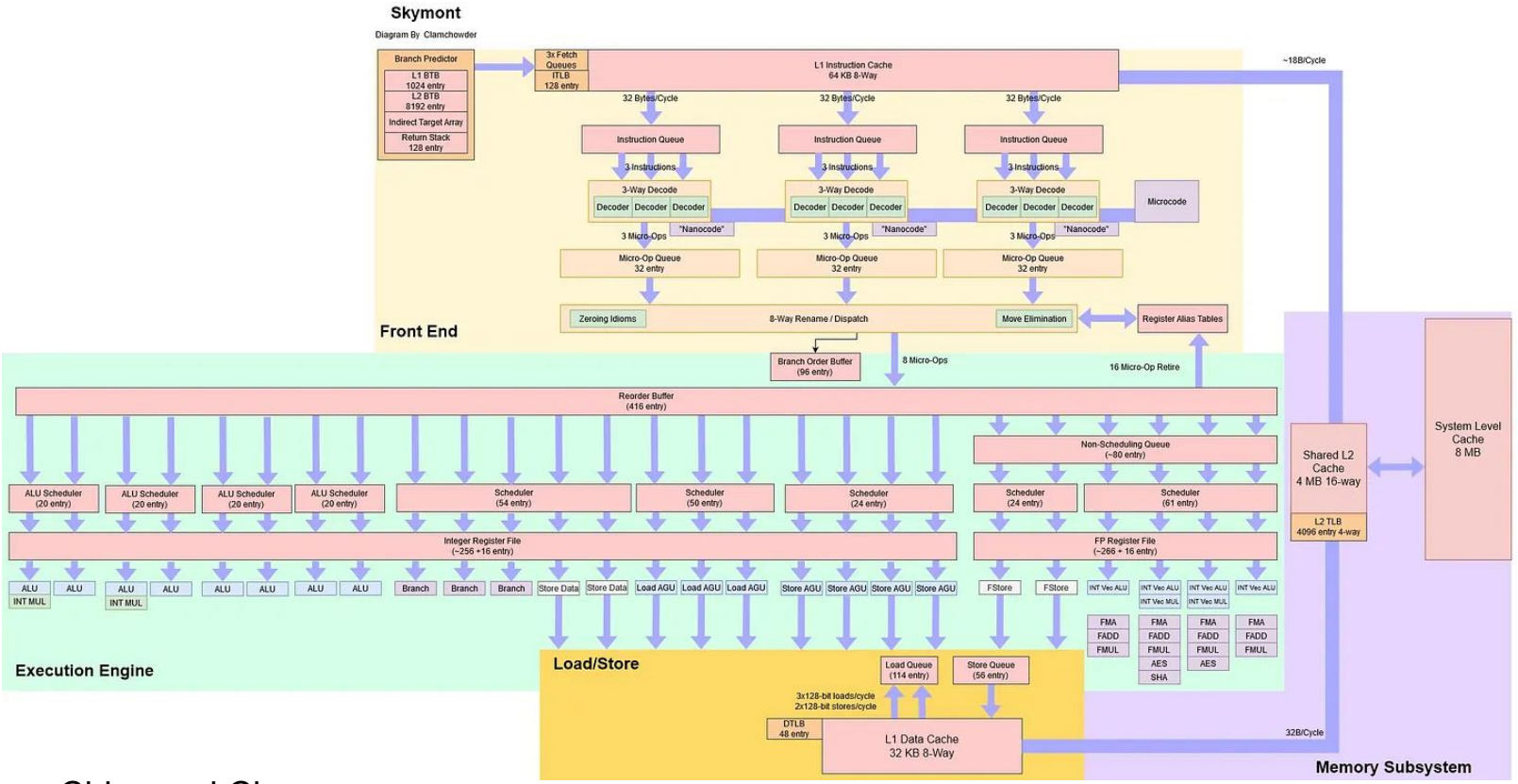
Comparison to E cores (Skymont)

Lion Cove (P Core)	Skymont (E Core)
1x - 8 Wide decode stage	3x - 3 Wide decode stage
uOP cache (x86 to uOP translation)	**No uOP cache**
192 entry uOP queue	3x - 32 entry uOP queues
12 wide retirement	16 wide retirement
4x 256 bit SIMD per cycle	4x 128 bit SIMD per cycle
L0, L1, L2, + Shared L3	**L1 Cache + Shared L2**

How does the OS know which core to use?

The thread director communicates to the OS to give it metrics to better schedule threads across P and E cores.

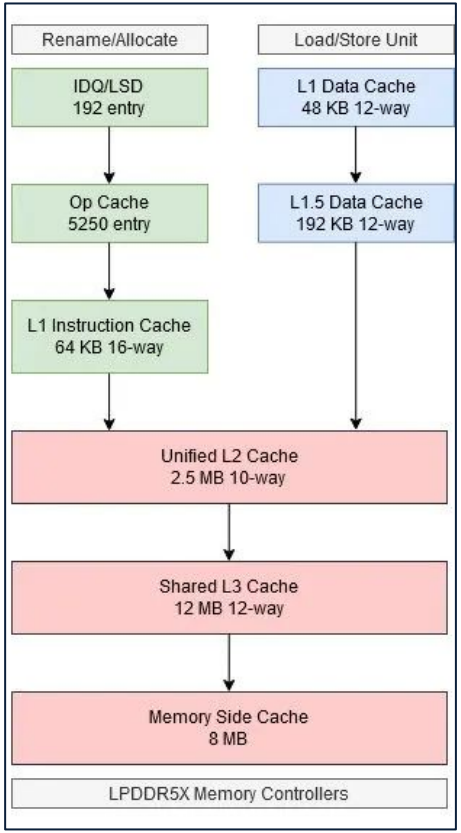
E cores (Skymont)



Source: Chips and Cheese

Cache hierarchy-Lion Cove

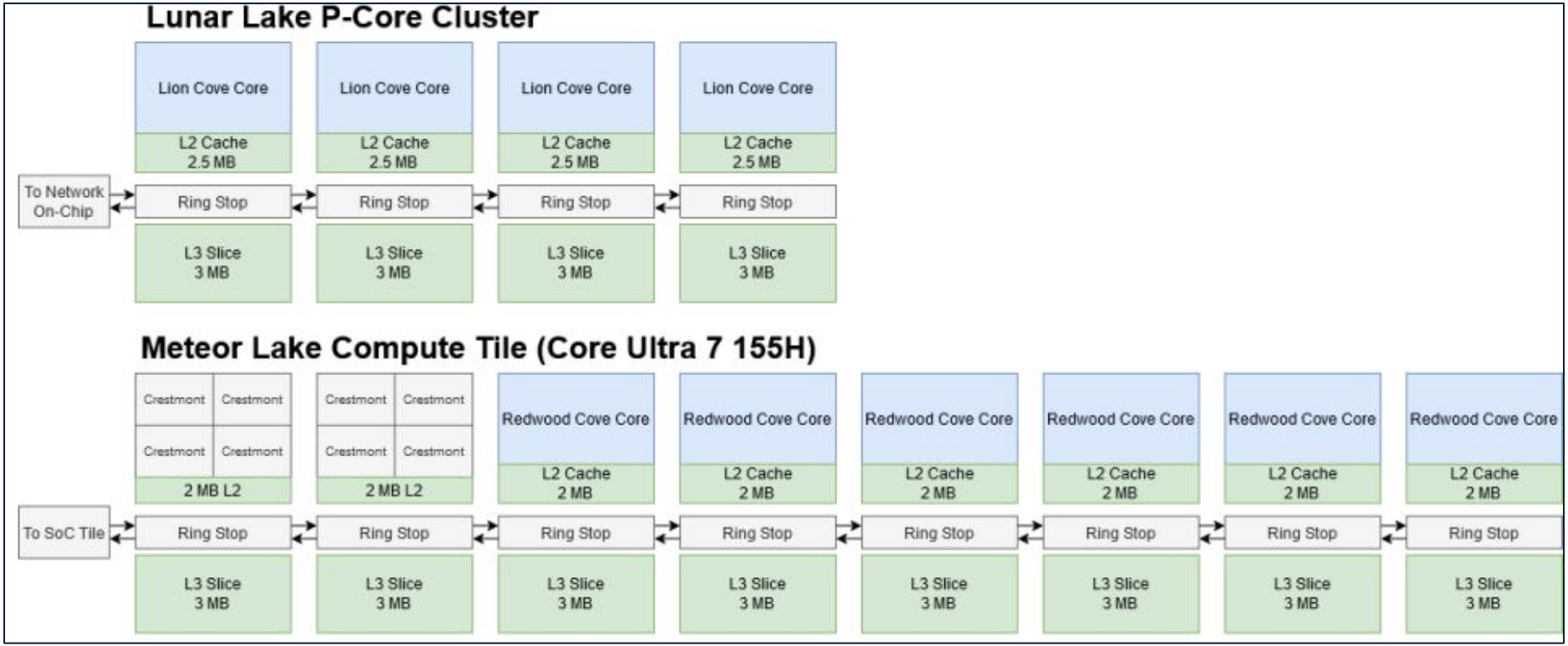
- L0 Data Cache: A 48 KB cache with a load-to-use latency of 4 cycles.
- L1 Data Cache: A 192 KB cache with a load-to-use latency of 9 cycles.
- L2 Cache: Each P-core has a private 2.5 MB L2 cache in the low-power implementation. The desktop implementation has a 3 MB L2 cache.
- L3 Cache: The four P-cores share a 12 MB L3 cache.



L0 Data Cache

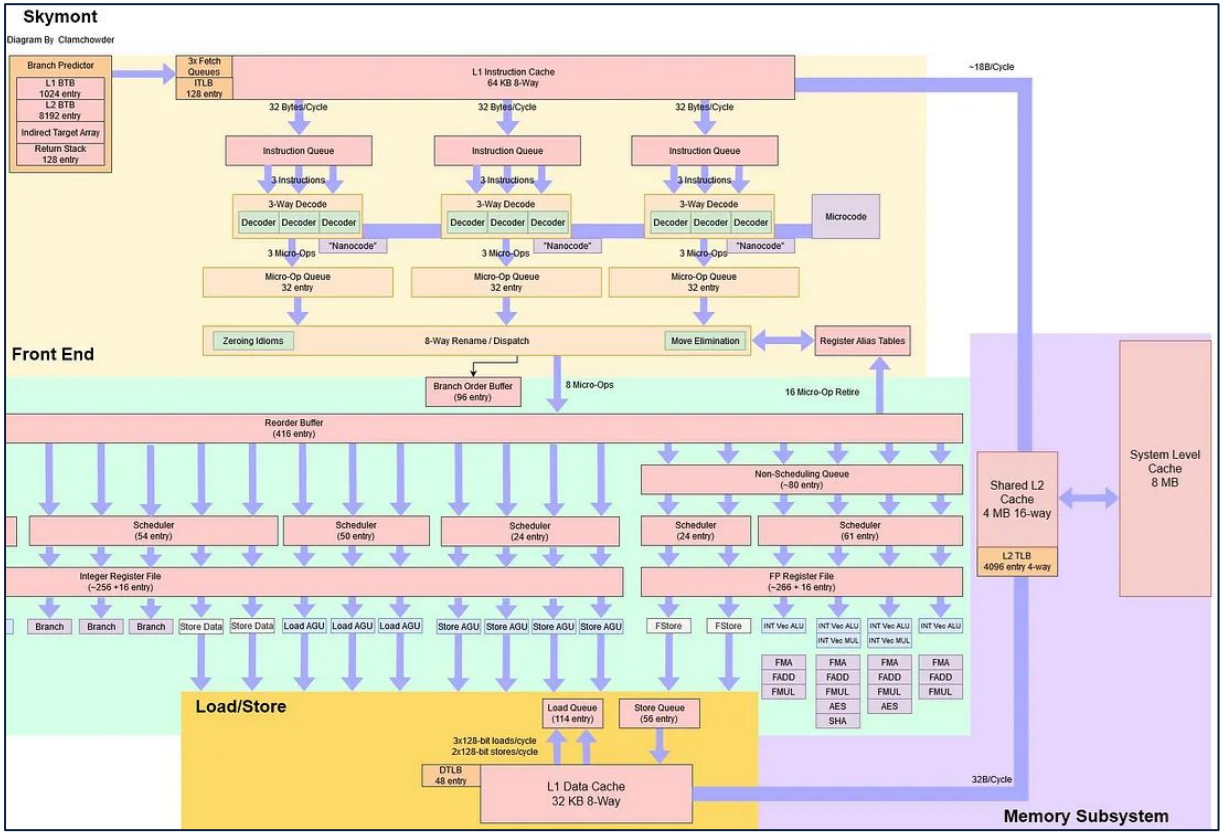
L1 Data Cache

Cache hierarchy-Lion Cove



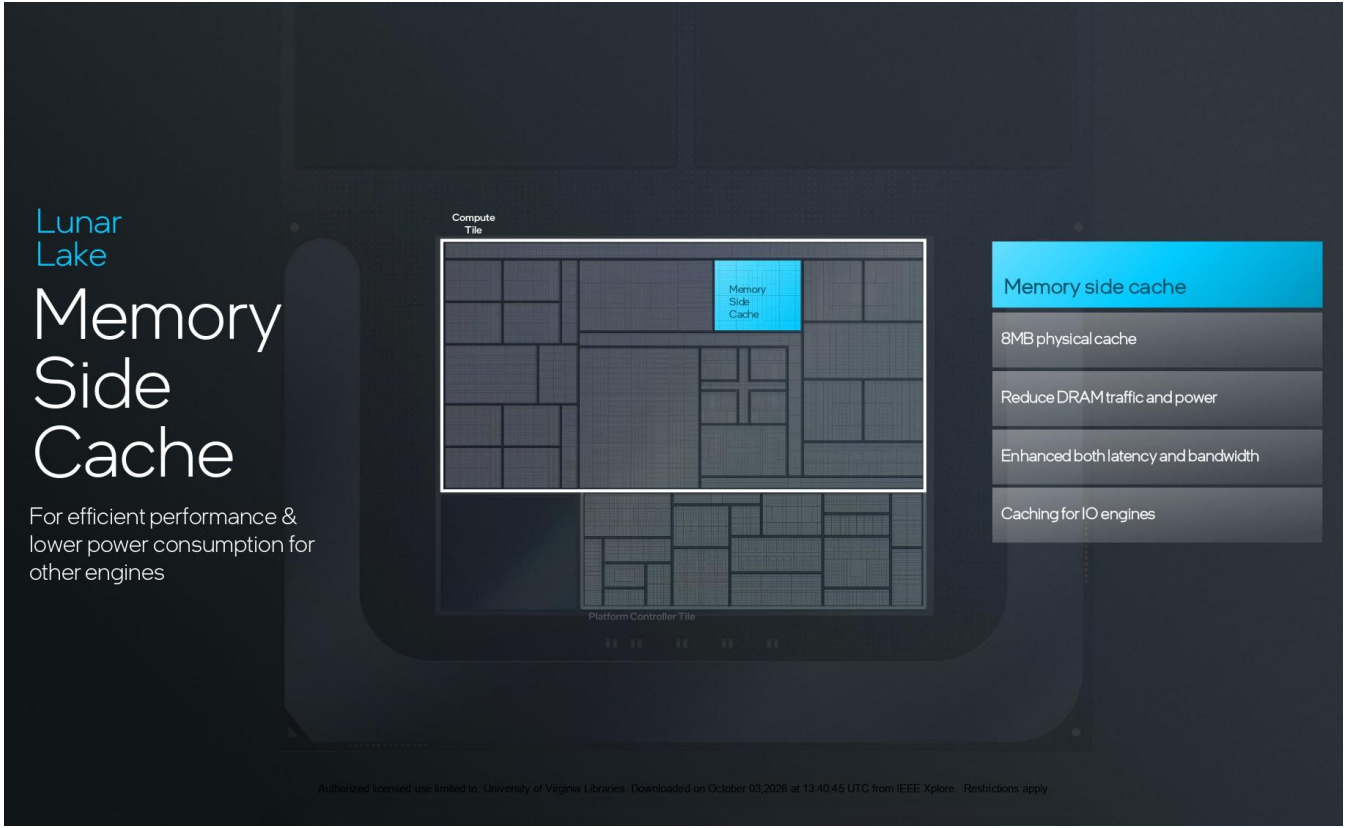
Cache hierarchy-Skymont

- L1 Data Cache: Each E-core has its own private **32 KB L1 data cache**.
- L2 Cache: The four E-cores form a unified Low-Power Island (LPI) cluster and **share a single 4 MB L2 cache**.
- L3 Cache: **None**. The E-core cluster does not directly access the 12 MB L3 cache, avoiding unnecessary activation of the power-intensive compute tile.



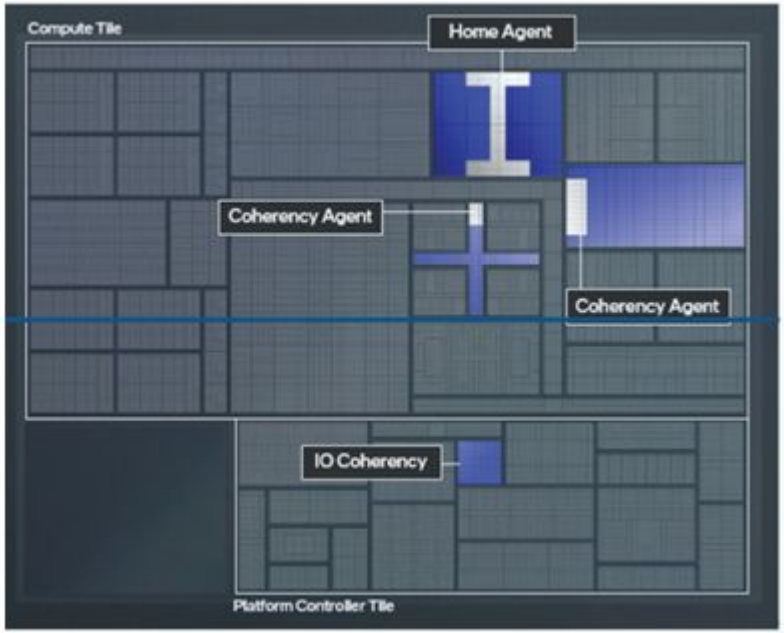
Cache hierarchy- System level side cache

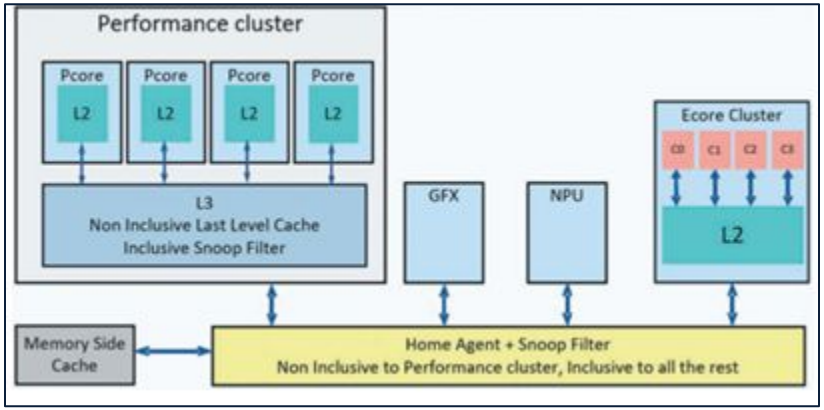
Slower than L0, L1, L2, L3 but can be used across various components.



Authorized licensed use limited to: University of Virginia Libraries. Downloaded on October 03,2026 at 13:40:45 UTC from IEEE Xplore. Restrictions apply.

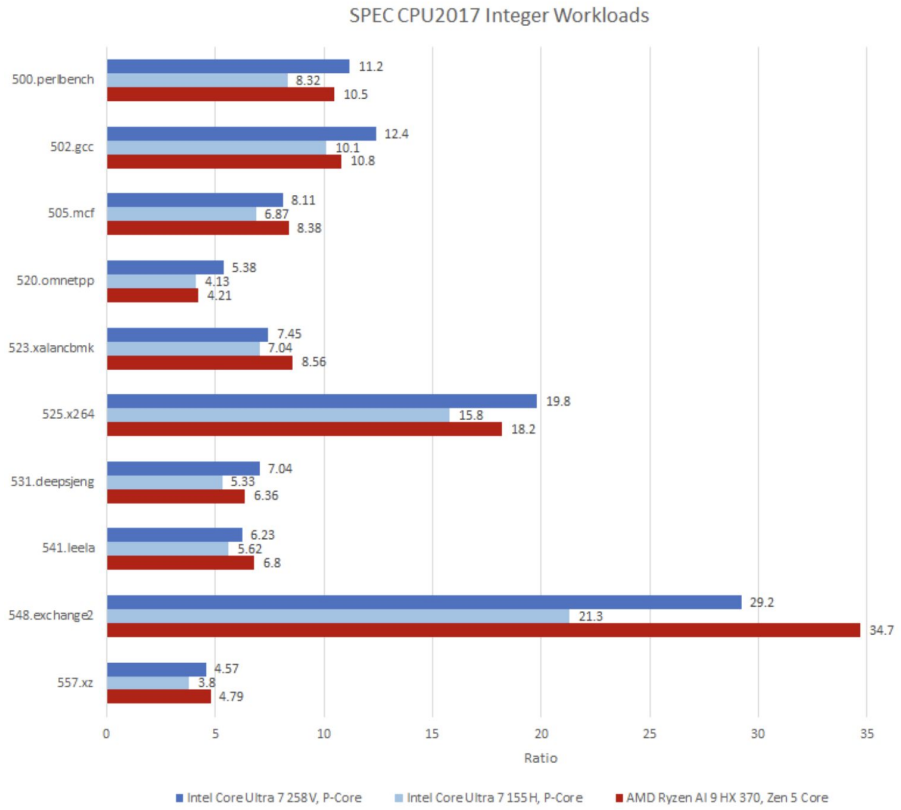
Lunar Lake - Compute And Coherency For Low-Power Design

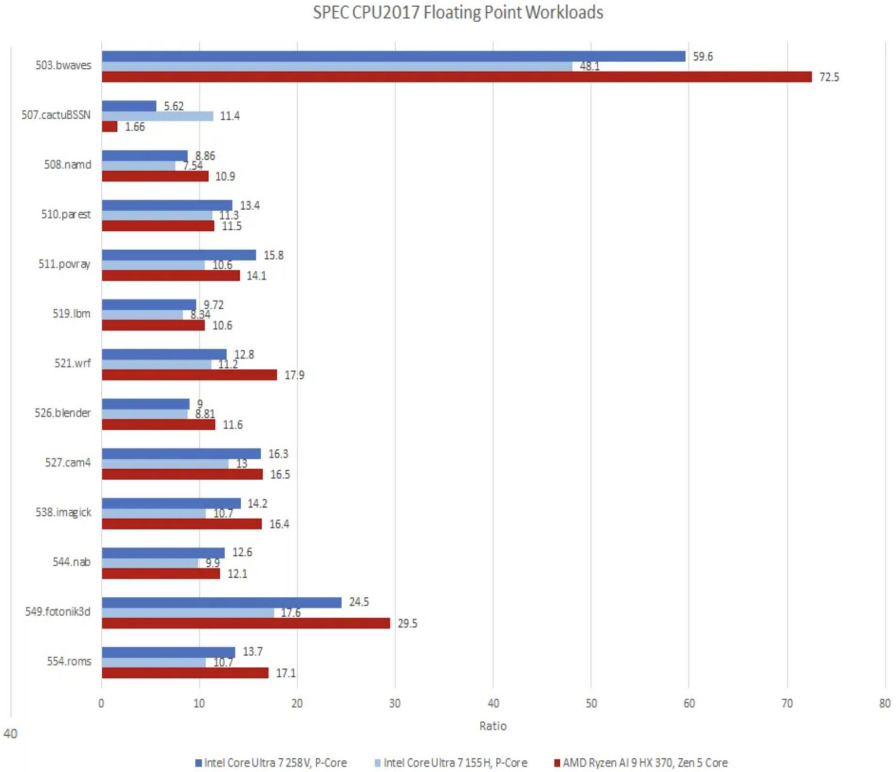




Coherency scheme

Benchmarks





Source: Chips and Cheese's independent analysis

Questions

Why did intel split the Integer and Floating point register files and schedulers?

Why did intel choose to cluster the E core decode and increase retire width compared to P cores?

Can mobile x86 chips like this one match ARM’s power consumption and performance?

Why retire 12 uOPs per cycle but dispatch only 8 uOPs per cycle?

What are the advantages and disadvantages of putting RAM directly on the package?

Thank You

Sources / Dossier :
https://sai2131.github.io/IntelLunarLake